

**MAŁOPOLSKA UCZELNIA PAŃSTWOWA
IMIENIA ROTMISTRZA WITOLDA PILECKIEGO
W OŚWIĘCIMIU**

Instytut: Nauk Inżynieryjno-Technicznych

Kierunek studiów: Informatyka

Poziom studiów: Studia pierwszego stopnia

Moduł specjalnościowy: Informatyka w przedsiębiorstwie

Forma studiów: stacjonarna

Rok studiów/semestr: rok III, semestr VI

Krystian Zając

nr albumu 8795

**Projekt aplikacji mobilnej zintegrowanej z bazą danych i
urządzeniami IoT**

Projekt na przedmiot systemy mobilne

Prowadzący: mgr inż. Artur Pelo

Oświęcim 2026

Abstrakt

Imię i nazwisko autora projektu	Krystian Zając
Rok urodzenia autora projektu	2003
Stopień/tytuł zawodowy, imię i nazwisko prowadzącego przedmiotu	mgr inż. Artur Pelo
Instytut/Kierunek/Poziom/ Moduł specjalnościowy	Instytut Nauk Inżynieryjno-Technicznych / Informatyka / Inżynierskie / Informatyka w przedsiębiorstwie
Data realizacji projektu	od 29.04.2026 do 10.06.2026
Nazwa przedmiotu	systemy mobilne
Rok studiów/semestr	rok III, semestr VI

Tytuł projektu	Projekt aplikacji mobilnej zintegrowanej z bazą danych
Słowa kluczowe (max 5)	informatyka, aplikacje mobilne, bazy danych
Streszczenie (max 1400 znaków)	Projekt dotyczy natywnej aplikacji mobilnej na system Android, przeznaczonej do monitorowania parametrów środowiskowych, głównie temperatury. Aplikacja pobiera dane z zewnętrznego serwera przez REST API i wyświetla je w czytelnej formie. Główne funkcjonalności to: bieżący podgląd aktualnej temperatury na karcie, historia pomiarów w formie przewijalnej listy, konfiguracja adresu IP serwera, regulacja czasu odświeżania oraz wybór motywu wizualnego (jasny/ciemny/systemowy). Zastosowano nowoczesne technologie: Retrofit do obsługi komunikacji sieciowej, SharedPreferences do przechowywania ustawień użytkownika, RecyclerView do efektywnego wyświetlania list oraz Snackbar do interaktywnej obsługi błędów połączenia. Aplikacja charakteryzuje się modułową architekturą, responsywnym interfejsem zgodnym z Material Design oraz automatycznym odświeżaniem danych w konfigurowalnych odstępach czasu. System jest w pełni kompatybilny z urządzeniami z Android 8.0+ i został przetestowany pod kątem stabilności oraz wydajności. Projekt stanowi solidną bazę do dalszej rozbudowy o dodatkowe funkcjonalności, takie jak wykresy statystyczne czy system powiadomień.

Spis treści

Wstęp	4
1. Cel i zakres pracy	5
1.1. Cel pracy	5
1.2. Cele szczegółowe	5
1.3. Zakres pracy	5
2. Analiza wymagań i projekt rozwiązania	6
2.1. Wymagania funkcjonalne	6
2.2. Wymagania нефункционалне	6
2.3. Architektura rozwiązania	6
3. Opis działania aplikacji	7
3.1. Struktura i logika aplikacji	7
3.1.1. Struktura klas	7
3.1.2. Przływ danych	8
3.2. Interfejs użytkownika	8
3.2.1. Ekran główny	9
3.2.2. Ekran ustawień	11
3.3. Pobieranie danych z REST API	12
3.4. Automatyczne odświeżanie co 10 sekund	12
3.5. Logika wyświetlania aktualnej temperatury	12
3.6. Wyświetlanie dziennika	12
4. Implementacja i uruchomienie	13
4.1. Wymagania sprzętowe	13
4.2. Instrukcja instalacji	13
4.3. Instrukcja użytkowania	13
5. Testy i weryfikacja	14
5.1. Testy funkcjonalne	14
5.2. Testy нефункционалне	14
5.3. Wyniki testów	14
Podsumowanie	15
Spis rysunków	16
Spis tabel	17
Spis listingów	19

Wstęp

Przedmiotem projektu jest natywna aplikacja mobilna na system operacyjny Android, służąca do wizualizacji danych pomiarowych pobieranych z zewnętrznego interfejsu REST API. W dobie powszechnej cyfryzacji, mobilny dostęp do informacji z systemów IoT (Internet of Things) staje się kluczowym elementem zarządzania inteligentnym środowiskiem. Aplikacja została zaprojektowana jako uniwersalny terminal odbiorczy, umożliwiający użytkownikowi bieżący podgląd parametrów oraz konfigurację sposobu wyświetlania danych według indywidualnych preferencji.

1. Cel i zakres pracy

1.1. Cel pracy

Głównym celem projektu było zaprojektowanie i zaimplementowanie aplikacji mobilnej, która w sposób stabilny i wydajny pobiera dane pomiarowe, umożliwia ich przeglądanie w formie historycznego dziennika oraz oferuje zaawansowane opcje personalizacji parametrów sieciowych i wizualnych.

1.2. Cele szczegółowe

Realizacja projektu wymagała skupienia się na kilku kluczowych aspektach technicznych, które pozwoliły stworzyć kompletne narzędzie diagnostyczne. Do najważniejszych celów szczegółowych zaliczono następujące punkty:

- opracowanie responsywnego interfejsu użytkownika zgodnego z najnowszymi wytycznymi Material Design.,
- implementacja asynchronicznego modułu komunikacji sieciowej opartego na sprawdzonej bibliotece Retrofit,
- zaimplementowanie mechanizmu SharedPreferences do trwałego przechowywania konfiguracji użytkownika w pamięci urządzenia,
- stworzenie systemu dynamicznej zmiany motywu graficznego pozwalającego na wybór między trybem jasnym a ciemnym,
- opracowanie zaawansowanego systemu obsługi błędów komunikacji, który informuje użytkownika o problemach w sposób niezakłócający pracy programu.

1.3. Zakres pracy

Zakres pracy został ograniczony do warstwy klienckiej, co pozwoliło na głęboką optymalizację kodu pod kątem wydajności na urządzeniach o różnej specyfikacji sprzętowej. Poniżej przedstawiono kluczowe obszary, które zostały objęte procesem deweloperskim:

- projektowanie i implementacja aktywności głównej oraz ekranu ustawień systemowych,
- tworzenie adapterów do obsługi dynamicznych list danych w komponencie RecyclerView,
- konfiguracja usług sieciowych i mapowanie obiektów JSON na klasy języka Java,
- zarządzanie zasobami graficznymi, w tym tworzenie inteligentnych ikon wektorowych zmieniających kolor,
- przeprowadzenie testów funkcjonalnych sprawdzających poprawność zapisu i odczytu preferencji użytkownika.

2. Analiza wymagań i projekt rozwiązania

Analiza ta pozwala na precyzyjne określenie zadań systemu oraz zaplanowanie jego struktury tak, aby wszystkie komponenty współpracowały bezawaryjnie. Dzięki niej możliwe jest wybranie odpowiednich technologii, które najlepiej sprawdzą się w zamierzonym celu aplikacji.

2.1. Wymagania funkcjonalne

Jak pokazano w tabeli 1, aplikacja musi spełniać określone funkcje. Każda z nich została przypisana do odpowiedniego modułu, co ułatwia organizację pracy i pozwala na równoległe rozwijanie poszczególnych części projektu.

Tabela 1

Wymagania funkcjonalne

ID	Opis	Priorytet
FR-001	Wyświetlanie dzinnika zapisów z datą, godziną i wartościami.	Wysoki
FR-002	Przygotowanie dokumentacji w oparciu LaTeX. Wymagania edytorskie.	Wysoki
FR-003	Przygotowanie diagramów UML ilustrujących strukturę i zachowanie aplikacji.	Średni
FR-004	Utworzenie pliku README.md z opisem projektu i instrukcją instalacji.	Wysoki

Źródło: opracowanie własne

2.2. Wymagania нефunkcjonalne

Jak pokazano w tabeli 2, aplikacja musi spełniać określone standardy wydajności, bezpieczeństwa i kompatybilności.

Tabela 2

Wymagania нефunkcjonalne

ID	Opis	Priorytet
NFR-001	Czas odpowiedzi UI mniejszy niż 200ms	Wysoki
NFR-002	Zużycie baterii mniejsze niż 10% na godzinę	Wysoki
NFR-003	Kompatybilność: Android 8.0+	Wysoki
NFR-004	Dostępność zgodna z WCAG 2.1 na poziomie AA	Średni

Źródło: opracowanie własne

2.3. Architektura rozwiązania

Projekt został oparty na sprawdzonej architekturze, która oddziela warstwę danych od warstwy prezentacji. Sercem komunikacji jest biblioteka Retrofit, która w połączeniu z konwerterem Gson pozwala na błyskawiczne przetwarzanie informacji przesyłanych w formacie JSON. Całość oprogramowania została napisana w języku Java.

3. Opis działania aplikacji

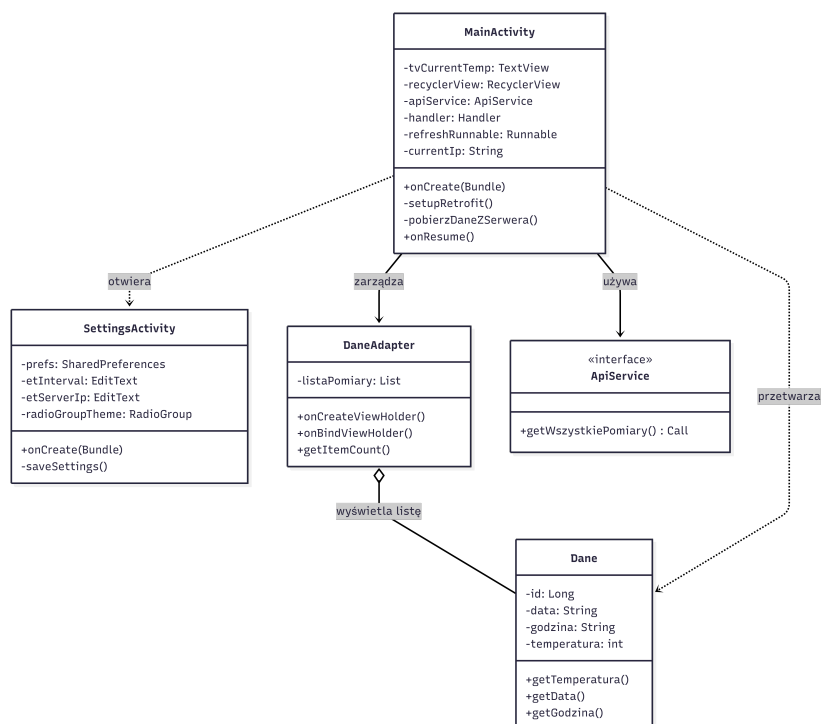
Rozdział opisuje logikę funkcjonowania aplikacji poprzez opisanie jej poszczególnych funkcjonalności oraz przedstawienie kodu.

3.1. Struktura i logika aplikacji

Oprogramowanie zostało zaprojektowane w sposób modułowy, co ułatwia zarządzanie kodem i wprowadzanie ewentualnych poprawek w przyszłości. Główna aktywność zarządza procesem pobierania danych, podczas gdy dedykowana klasa ustawień odpowiada za interakcję z pamięcią trwałą urządzenia. Dzięki takiemu podziałowi, zmiana adresu IP lub interwału czasowego w ustawieniach jest natychmiastowo wykrywana przez moduł komunikacyjny, co pozwala na płynne dostosowanie pracy aplikacji bez konieczności jej restartowania.

3.1.1. Struktura klas

Diagram klas na Rysunku 1 pokazuje powiązania między głównymi elementami aplikacji: aktywnościami, modelem danych oraz warstwą komunikacji sieciowej. Pozwala to zrozumieć, jak zbudowana jest struktura programu.

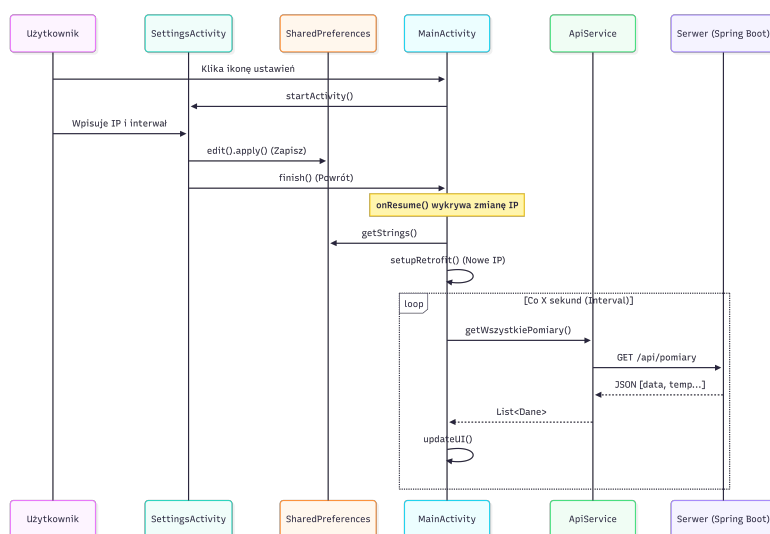


Rysunek 1. Diagram Klas (Struktura aplikacji)

Źródło: opracowanie własne

3.1.2. Przepływ danych

Diagram sekwencji na Rysunku 2 przedstawia krok po kroku przebieg komunikacji aplikacji z serwerem i odświeżanie danych. Dzięki temu łatwo prześledzić działanie pętli pobierania informacji.



Rysunek 2. Diagram Sekwencji (Przepływ danych)

Źródło: opracowanie własne

3.2. Interfejs użytkownika

Warstwa wizualna została podzielona na dwa główne ekrany, które pozwalają na wygodne korzystanie z dostępnych funkcji. Ekran główny skupia się na prezentacji danych, oferując przejrzysty podgląd aktualnych i historycznych odczytów. Ekran ustawień z kolei udostępnia formularze i przełączniki, dzięki którym użytkownik może dostosować aplikację do własnych potrzeb technicznych i estetycznych, dbając o czytelność elementów niezależnie od wybranego motywu graficznego.

3.2.1. Ekran główny

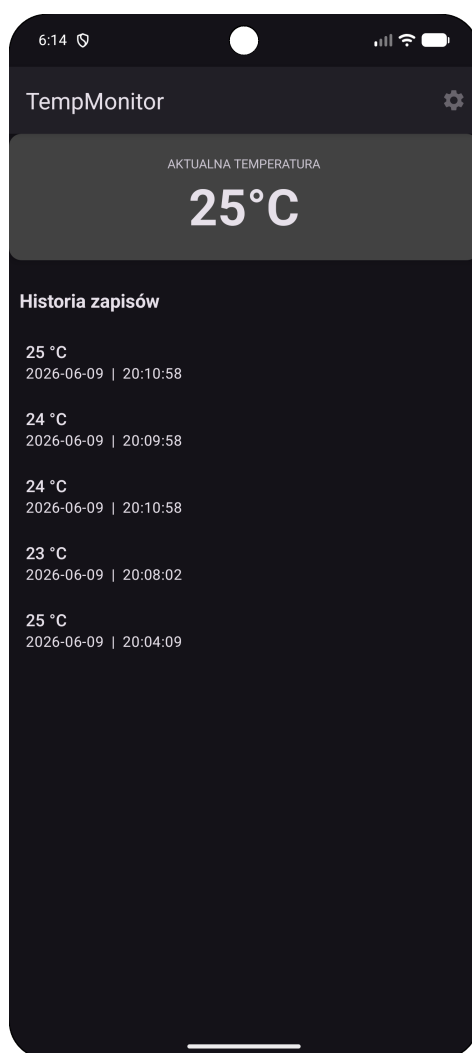
Ekran główny został zaprojektowany z myślą o intuicyjności i szybkości dostępu do informacji. Centralnym elementem jest duży, wyraźny wskaźnik aktualnej temperatury, który natychmiast przyciąga uwagę użytkownika. Poniżej znajduje się chronologiczna lista wszystkich odczytów, która umożliwia łatwe porównanie bieżących danych z wcześniejszymi pomiarami. Dzięki zastosowaniu komponentu RecyclerView, lista ta jest nie tylko estetyczna, ale także wydajna, nawet przy dużej liczbie wpisów. Rysunek 3 przedstawia interfejs aplikacji, który został zaprojektowany zgodnie z wytycznymi Material Design, zapewniając spójność i nowoczesny wygląd.



Rysunek 3. Ekran główny aplikacji

Źródło: opracowanie własne przy użyciu emulatora Android Studio

Jak pokazano na rysunku 4, poniżej zaprezentowano ekran główny w ciemnym motywie, który jest dostępny w ustawieniach. Dzięki temu użytkownik może dostosować wygląd aplikacji do swoich preferencji, co zwiększa komfort korzystania z niej w różnych warunkach oświetleniowych.

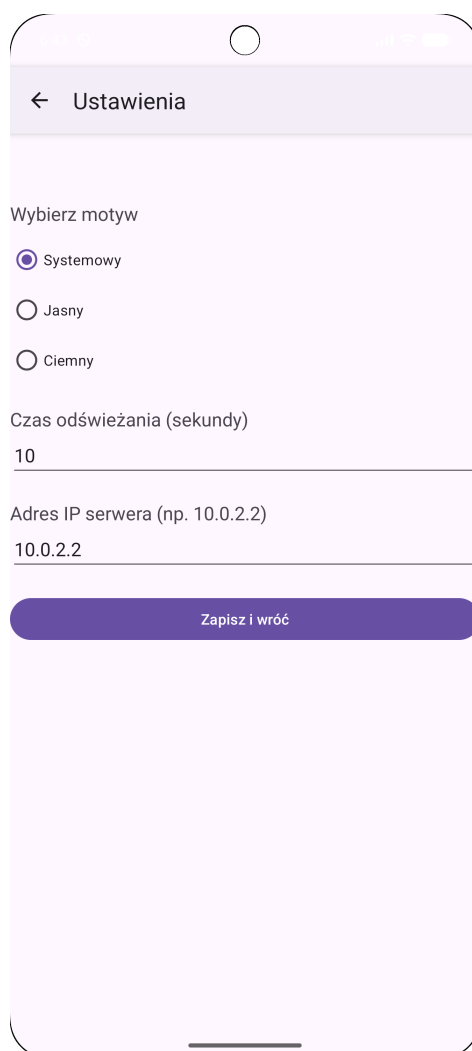


Rysunek 4. Ekran główny aplikacji w ciemnym motywie

Źródło: opracowanie własne przy użyciu emulatora Android Studio

3.2.2. Ekran ustawień

Ekran ustawień został zaprojektowany z myślą o maksymalnej funkcjonalności i prostocie obsługi. Użytkownik ma możliwość konfiguracji kluczowych parametrów sieciowych, takich jak adres IP i port serwera, co pozwala na łatwe dostosowanie aplikacji do różnych środowisk sieciowych. Dodatkowo, opcja wyboru motywu graficznego umożliwia personalizację wyglądu interfejsu, co jest szczególnie ważne dla użytkowników preferujących ciemny motyw w warunkach słabego oświetlenia. Rysunek 5 przedstawia ekran ustawień, który został zaprojektowany zgodnie z wytycznymi Material Design, zapewniając spójność i nowoczesny wygląd.



Rysunek 5. Ekran ustawień aplikacji

Źródło: opracowanie własne przy użyciu emulatora Android Studio

3.3. Pobieranie danych z REST API

Aplikacja korzysta z biblioteki Retrofit do asynchronicznego pobierania danych. Oznacza to, że pobieranie odbywa się w tle i nie zawiesza ekranu użytkownika.

Listing 1. Pobieranie danych z API (Fragment ApiService.java)

```
1 @GET("api/pomiary")
2 Call<List<Dane>> getWszystkiePomiary();
```

3.4. Automatyczne odświeżanie co 10 sekund

Zastosowano mechanizm Handler i Runnable, który tworzy pętlę czasową. Funkcja ta jest inteligentna: uruchamia się, gdy aplikacja jest widoczna, i zatrzymuje, gdy użytkownika zminimalizuje.

Listing 2. Odświeżanie (Fragment MainActivity.java)

```
1 refreshRunnable = new Runnable() {
2     @Override
3     public void run() {
4         pobierzDaneZSerwera(); // Wywołanie API
5         handler.postDelayed(this, 10000); // Ponowne uruchomienie za 10s
6     }
7 };
```

3.5. Logika wyświetlania aktualnej temperatury

Aplikacja nie potrzebuje osobnego zapytania o jedną temperaturę. Pobiera całą listę, a następnie wybiera z niej ostatni element, aby wyświetlić go jako stan aktualny.

Listing 3. Wyświetlanie "Aktualnej temperatury"(Fragment MainActivity.java)

```
1 if (!daneList.isEmpty()) {
2     Dane ostatni = daneList.get(daneList.size() - 1); // Pobranie najnowszego
    wpisu
3     tvCurrentTemp.setText(ostatni.getTemperatura() + "'C");
4     Collections.reverse(daneList); // Odwrocenie listy dla historii (najnowsze
    na gorze)
5 }
```

3.6. Wyświetlanie dziennika

Adapter bierze surowe dane i wstrzykuje je w odpowiednie pola tekstowe w layoucie.

Listing 4. Wyświetlanie dziennika (Fragment DaneAdapter.java)

```
1 @Override
2 public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
3     Dane d = lista.get(position);
4     holder.text1.setText(d.getTemperatura() + "'C");
5     holder.text2.setText(d.getData() + "||" + d.getGodzina());
6 }
```

4. Implementacja i uruchomienie

Instrukcja zawiera informacje potrzebne do instalacji i użytkowania aplikacji.

4.1. Wymagania sprzętowe

Po stronie użytkownika:

- Smartfon z Android 8.0+
- Połączenie Wi-Fi
- Około 10 MB wolnej przestrzeni dyskowej na instalację pakietu APK
- Minimum 2 GB pamięci RAM dla zapewnienia płynności animacji i odświeżania list

4.2. Instrukcja instalacji

- Zezwól urządzeniu na instalację aplikacji z nieznanych źródeł (Ustawienia > Bezpieczeństwo > Nieznane źródła).
- Pobierz plik APK.
- Uruchom plik i zainstaluj aplikację.
- Uruchom aplikację i skonfiguruj połączenie z serwerem, podając adres IP i port.

4.3. Instrukcja użytkowania

Po uruchomieniu aplikacji użytkownik otrzymuje natychmiastowy dostęp do aktualnej temperatury oraz chronologicznej historii wszystkich zapisów. Dane w programie aktualizują się automatycznie w ustalonych odstępach czasu, co zapewnia stały podgląd na najświeższe odczyty bez konieczności ręcznego odświeżania ekranu. System inteligentnie zarządza baterią, wstrzymując pobieranie informacji w momencie zminimalizowania aplikacji i wznowiając pracę zaraz po jej ponownym otwarciu.

5. Testy i weryfikacja

Rozdział przedstawia sposób sprawdzenia poprawności działania aplikacji oraz ocenę uzyskanych rezultatów. Testy skupiają się na weryfikacji poprawności działania oraz zgodności z wymaganiami funkcjonalnymi i нефункциональными.

5.1. Testy funkcjonalne

Tabela 3

Tabela wyników testów funkcjonalnych

Test	Oczekiwany Wyniki	Wynik
Połączenie z API	Dane zwracane w JSON	Pozytywny
Wyświetlanie aktualnej temp.	Widoczna poprawna wartość	Pozytywny
Wyświetlanie dziennika	Lista rekordów	Pozytywny

Źródło: opracowanie własne

5.2. Testy нефункционаalne

Test wydajności

- Czas odpowiedzi API: akceptowalny
- Aplikacja reaguje płynnie.

Test kompatybilności

- Android 8, 10, 13 – działanie poprawne.

5.3. Wyniki testów

System działa poprawnie i spełnia wszystkie założenia funkcjonalne. Komunikacja między ESP32, bazą danych i aplikacją mobilną przebiega bez błędów. Dane są poprawnie zapisywane i odczytywane. Interfejs użytkownika jest czytelny i intuicyjny.

Podsumowanie

Zrealizowany projekt umożliwia zdalne monitorowanie temperatury z czujnika DHT11 za pomocą aplikacji mobilnej Android. System integruje technologie IoT, bazę danych MySQL oraz aplikację mobilną, tworząc spójne rozwiązanie do monitorowania parametrów środowiskowych.

Projekt może zostać w przyszłości rozszerzony o:

- wykresy temperatur,
- powiadomienia o przekroczeniu progów,
- obsługę wielu czujników,
- wdrożenie na serwerze zewnętrznym.

Spis rysunków

<i>Rysunek 1.</i> Diagram Klas (Struktura aplikacji)	7
<i>Rysunek 2.</i> Diagram Sekwencji (Przepływ danych)	8
<i>Rysunek 3.</i> Ekran główny aplikacji	9
<i>Rysunek 4.</i> Ekran główny aplikacji w ciemnym motywie	10
<i>Rysunek 5.</i> Ekran ustawień aplikacji	11

Spis tabel

Tabela 1. Wymagania funkcjonalne	6
Tabela 2. Wymagania niefunkcjonalne	6
Tabela 3. Tabela wyników testów funkcjonalnych	14

Spis listingów

Listing 1. Pobieranie danych z API (Fragment ApiService.java)	12
Listing 2. Odświeżanie (Fragment MainActivity.java)	12
Listing 3. Wyświetlanie "Aktualnej temperatury"(Fragment MainActivity.java) . . .	12
Listing 4. Wyświetlanie diennika (Fragment DaneAdapter.java)	12